

BEGINNER LESSON PLAN

Learn JavaScript by Building a Stealth Dungeon Game

Based on your Lesson_plan.js (gameCanvas3), plus torches, shadows, throwables, torch patrols and modular rooms with cover.

Part	Contents
Part 1	Lessons 1-12: JavaScript foundations taught through your game
Part 2	Lessons 13-17: stealth lighting, shadows, throwing, torch patrols, modular rooms and cover
Part 3	Integration checklist, debugging habits and capstone ideas
Appendices	Bug hunt, function map, glossary, answer key

Before you start

About this plan

This plan teaches beginner JavaScript using your own game, `Lesson_plan.js` (the `gameCanvas3` dungeon stealth game). Every lesson points at real functions in your file, so you learn the language by reading, changing and breaking code you already care about. Part 1 (Lessons 1-12) covers core JavaScript. Part 2 (Lessons 13-17) teaches the next skills by adding the stealth features you asked for: torches and lighting, shadows, throwables, torch-carrying patrols and modular rooms with cover.

The learning loop used in every lesson

Read the code named in the lesson. **Predict** what a change will do. **Change** it and run the game. **Break** it on purpose and read the error. **Fix** it. Always work on a copy of the file and keep the original safe.

Setup

A minimal HTML page that runs your file (the script must come after the canvas)

```
<!DOCTYPE html>
<html>
  <body>
    <canvas id="gameCanvas3"></canvas>
    <script src="Lesson_plan.js"></script>
    <script>startGame3();</script>
  </body>
</html>
```

Open the page in a browser, press F12 and look at the **Console** tab. Errors appear in red with a file name and line number. The sprites load from GitHub, so you need an internet connection. Movement is W A S D, C sneaks, Left Shift runs, E interacts, Tab opens the inventory, Q zooms out, M toggles the minimap.

Course map

Lesson	Topic	Where in your file	Time
1	Reading a real program	whole file, section banners	45 min
2	Variables and constants	<code>TILE_SIZE</code> , <code>PLAYER_SPEED</code>	45 min
3	Data types and structures	<code>TileType</code> , <code>gameState</code> , <code>Set</code> , <code>Map</code>	60 min
4	Functions	<code>isSolidTile</code> , <code>rollInt</code> , <code>steerToward</code>	60 min
5	Decisions and loops	<code>drawMap</code> , speed ternary	60 min
6	Events and input	keydown handlers, mouse aim	60 min
7	The game loop and time	<code>gameLoop</code> , delta time	60 min
8	Canvas drawing	<code>drawGame</code> , <code>drawPlayer</code>	60 min
9	Arrays and objects in depth	<code>filter</code> , <code>find</code> , <code>reduce</code> , <code>??=</code>	60 min
10	Collision and movement	<code>isBoxClear</code> , <code>tryMove</code>	60 min
11	AI state machines and pathfinding	<code>updateEnemyAI</code> , <code>findPath</code>	90 min
12	Data-driven design and generation	<code>CONTAINER_TYPES</code> , <code>createGameState</code>	90 min
13	Torches and lighting	new: light sources, darkness layer	90 min
14	Shadows and crouch hiding	new: <code>isPlayerInLight</code>	60 min
15	Throwing: stones, torches, water	new: <code>throwItem</code> , <code>landThrown</code>	90 min
16	Torch patrols across the dungeon	new: <code>makeDungeonPatrollers</code>	90 min
17	Modular rooms, loops and cover	new: loop connectors, <code>placeCover</code>	120 min

Suggested pace: two sessions a week for eight weeks. Lessons 1-8 in weeks 1-4, lessons 9-12 in weeks 5-6, lessons 13-17 in weeks 7-8 (one per half week), then the capstone.

Part 1 - JavaScript foundations through your game

LESSON 1

Reading a real program

45 min

Goal: Run the game and build a map of the file before changing anything.

What you will learn

- A program is a text file of instructions the browser runs from top to bottom.
- Comments (`//`) and banner comments (`// -----`) are notes for humans.
- `"use strict"` turns silent mistakes (such as using an undeclared variable) into errors.
- The `{ ... }` block wrapping your file keeps every name private. Only `window.startGame3` and `window.stopGame3` are shared with the page.
- The console and `console.log()` are your main debugging tools.

Find it in your file

- Line 1 `"use strict";` and the opening `{`.
- `document.getElementById("gameCanvas3")` connects the code to the `<canvas>` in your HTML.
- Banner comments: STAIRS, TRAPS, LAYOUT / DOORS, ENEMY VISION, HIDING, INPUT, CORE GAME LOOP, PLAYER MOVEMENT, NOISE, DRAWING, ENEMY AI, DOORS, CONTAINERS, INVENTORY, SPELLS, ZOOM, ENEMY SPAWNING, LEVEL GENERATION.
- The last lines: `window.startGame3 = startGame3;`

Try it

- Run the page. Add `console.log("hello from my game");` as the first line of `startGame3`, reload, and find it in the console.
- Write one line per banner section in a notebook: what do you think it does? Revisit this list after Lesson 12.
- Break it: move the `<script src>` above the `<canvas>` and read the error.

Check yourself

1. Why is the whole file wrapped in a block?
2. What goes wrong if the script loads before the canvas exists?

LESSON 2

Variables and constants

45 min

Goal: Tell `const` from `let`, and replace magic numbers with named values.

What you will learn

- `const` cannot be reassigned; `let` can. Prefer `const` and use `let` only for values that change.
- `UPPER_SNAKE_CASE` names are tuning knobs; camelCase names are ordinary variables.
- A magic number is an unexplained literal. A named constant explains itself and is changed in one place.
- Constants can be built from other constants (`DISPLAY_WIDTH`). Template literals build strings with `${}`.

Find it in your file

- `TILE_SIZE`, `MAP_WIDTH`, `VIEW_TILES_X`, `DISPLAY_WIDTH`.
- `PLAYER_SPEED`, `PLAYER_SNEAK_SPEED`, `PLAYER_RUN_SPEED`, `STAMINA_MAX`.
- `canvas.style.width` is set with a template literal: a string in backticks containing `${...}`.
- `let gameRunning = false`; a value that changes while playing.

Constants versus variables

```
const PLAYER_SPEED = 3;           // tiles per second
const PLAYER_RUN_SPEED = 5.2;
const STAMINA_MAX = 3;
let gameRunning = false;         // let: changes while the game runs
```

Try it

- Predict, then set `PLAYER_RUN_SPEED` to 8. Is it fun? Set it back.
- Do not change `TILE_SIZE`: the sprites are 32x32 pixels.
- Add `const DEBUG_SPEED_BOOST = 2`; and multiply it into the walking speed. Then delete it.
- Try assigning to a `const` in the console of a copy and read the error.

Check yourself

1. Why is `gameRunning` a `let` but `TILE_SIZE` a `const`?
2. At 5.2 tiles per second and 0.016 s per frame, how far does a running player move per frame?

LESSON 3

Data types and structures

60 min

Goal: Recognise numbers, strings, booleans, arrays, objects, `Set` and `Map` and know when each is used.

What you will learn

- Primitive types: number, string, boolean, `null`, `undefined`.
- An array is an ordered list (`[]`); an object is a bundle of named values (`{}`).
- A `Set` holds unique values; a `Map` pairs keys with values.
- A 2D grid is an array of rows, so you read `map[y][x]` (row first).

Find it in your file

- `TileType` and `PLAYER_DIRECTION_ROWS` are objects used as lookup tables.
- `heldMoveKeys = new Set()` and `dungeons = new Map()`.
- `gameState` fields: `map`, `rooms`, `doors`, `player`, `enemies`, `items`, `containers`, `stairs`.
- `noiseRipples = []` is a plain array that is pushed to and spliced.

Using Set, Map and a 2D array

```
heldMoveKeys.add("a"); heldMoveKeys.has("a"); heldMoveKeys.delete("a");
dungeons.set(3, gameState); dungeons.get(3);
const tile = gameState.map[y][x]; // y (row) first, then x (column)
```

Try it

- Your code is hidden by the `{ }` block. Temporarily add `window.dbg = () => gameState;` just above `window.startGame3 = ...`. In the console type `dbg().player` and explore it.
- Type `dbg().player.hp = 10` and watch the HP bar react. Remove `dbg` when you finish the course.
- Find three different data types in `createGameState`'s returned object.

Check yourself

1. Why is it `map[y][x]` and not `map[x][y]`?
2. Why is `heldMoveKeys` a Set and not an array?

LESSON 4

Functions

60 min

Goal: Write small functions with parameters and return values, and read arrow functions.

What you will learn

- A function packages steps under a name. Parameters go in, a `return` value comes out; no `return` gives `undefined`.
- Default parameters: `function f(a, b = 5)`.
- Arrow functions are short functions: `(a, b) => a + b`. They are common as callbacks.
- Small single-purpose helpers make large programs readable.

Find it in your file

- `isSolidTile(t), canMoveTo(x, y), chargeTimeFor(type)`.
- `const rollInt = (lo, hi) => ...` and `const pickOne = (arr) => ...`.
- `steerToward(e, gx, gy, speed, dt, arriveDist = ENEMY_ARRIVE_DIST, faceMovement = true)` uses defaults.

Three styles of function from your file

```
function isSolidTile(t) {
  return t === TileType.WALL || t === TileType.PILLAR;
}
const rollInt = (lo, hi) => lo + Math.floor(Math.random() * (hi - lo + 1));
function chargeTimeFor(type) {
  return SCROLL_CHARGE_MS[type] ?? FIRE_COOLDOWN;
}
```

Try it

- Write `const clamp = (v, lo, hi) => Math.max(lo, Math.min(hi, v));` and use it inside `rechargeStamina` instead of `Math.min`.
- Write `distance(a, b)` returning `Math.hypot(a.x - b.x, a.y - b.y)` and test it with `console.log`.
- Call `rollInt(1, 6)` ten times and log the results.

Check yourself

1. What does a function without a `return` give back?
2. Why can you call a `function` declaration before the line that defines it?

Goal: Control the flow of a program with `if`, ternaries and loops.

What you will learn

- `if / else if / else`; comparisons with `===`; combine tests with `&&` (and), `||` (or), `!` (not).
- The ternary `a ? b : c` picks one of two values.
- `for`, `for...of` and `while` repeat code; `break` leaves a loop and `continue` skips to the next pass.
- Nested loops walk a grid.

Find it in your file

- `drawMap`: a loop over `y` containing a loop over `x`.
- The speed ternary chain in `updateFreePlayerMovement`.
- `for (const room of rooms)` in `placeHidingSpots`, and the shuffle at the end of its candidate loop.

A chained ternary and the shuffle loop

```
const speed =
  player.gait === "sneak" ? PLAYER_SNEAK_SPEED
  : player.gait === "run" ? PLAYER_RUN_SPEED
  : PLAYER_SPEED;

for (let i = candidates.length - 1; i > 0; i--) {
  const j = Math.floor(Math.random() * (i + 1));
  [candidates[i], candidates[j]] = [candidates[j], candidates[i]]; // swap
}
```

Try it

- After a dungeon is created, count floor tiles with two nested loops and `console.log` the total.
- Rewrite the speed ternary as an `if / else if / else` and check it behaves the same.
- In `drawMap`, temporarily replace a `continue` with `break` and see what disappears.

Check yourself

1. What does `continue` do inside the loops in `drawMap`?
2. In `a && b || c`, which part is evaluated first?

Goal: Respond to the keyboard and mouse without blocking the game loop.

What you will learn

- Events: the browser calls your function when something happens (`addEventListener`).
- `event.key` is the character; `event.code` is the physical key ("`KeyE`"), which is layout independent.
- `event.repeat` is true for auto-repeat after the first press.
- Held-key pattern: `keydown` adds to a `Set`, `keyup` removes, and the game loop reads the `Set` every frame.
- Always clear held keys on `blur`, or a key can stay "stuck" when the window loses focus.

Find it in your file

- The W A S D handlers using `heldMoveKeys` and `preventDefault()`.
- The `KeyE` handler (one-shot actions: hide, search, open door).
- `mouse`, `getMouseWorldTile()` and `getAimVector()` turning screen pixels into world tiles.
- Three separate `window.addEventListener("blur", ...)` calls (see the bug hunt).

Held-key pattern

```
window.addEventListener("keydown", (event) => {
  const key = event.key.toLowerCase();
  if (movementKeys.has(key)) { event.preventDefault(); heldMoveKeys.add(key); }
});
window.addEventListener("keyup", (event) => {
  const key = event.key.toLowerCase();
  if (movementKeys.has(key)) { event.preventDefault(); heldMoveKeys.delete(key); }
});
```

Try it

- Add a key (`KeyH`) that logs the player's HP.
- Create `const KEYS = { interact: "KeyE", sneak: "KeyC" };` and use it in one handler.
- Merge the duplicate `blur` listeners into one.

Check yourself

1. Why store held keys instead of moving on `keydown`?
2. Why prefer `event.code` for interact keys?

Goal: Understand `requestAnimationFrame`, *delta time* and *cooldowns*.

What you will learn

- The loop calls update then draw about 60 times a second, with a timestamp.
- Delta time (dt) is seconds since the last frame. Movement = speed x dt, so the game runs at the same speed on slow and fast screens.
- Clamping dt (here to 0.05) prevents a huge jump after the tab was in the background.
- Cooldowns compare timestamps (`performance.now()`) instead of counting frames.

Find it in your file

- `gameLoop(timestamp)`: the order of update calls, then `drawGame`, then the next `requestAnimationFrame`.
- `lastFrameTime` and `deltaTime` in `updateFreePlayerMovement`.
- `FIRE_COOLDOWN` and `lastFireTime`; `stopGame3` calling `cancelAnimationFrame`.

Delta time

```
const deltaTime =  
  lastFrameTime === null ? 0 : Math.min((timestamp - lastFrameTime) / 1000, 0.05);  
lastFrameTime = timestamp;  
const movementAmount = speed * deltaTime; // tiles moved this frame
```

Try it

- Add an FPS counter at the top of `gameLoop` and log it once a second:

FPS counter

```
let fpsFrames = 0, fpsStart = 0; // declare near the other let variables  
// first lines of gameLoop:  
fpsFrames++;  
if (timestamp - fpsStart >= 1000) {  
  console.log("fps:", fpsFrames); fpsFrames = 0; fpsStart = timestamp;  
}
```

Try it

- Remove `Math.min(..., 0.05)` in a copy, switch tabs for five seconds, come back, and describe what happens.
- Halve `FIRE_COOLDOWN` and fire repeatedly.

Check yourself

1. Why multiply speed by dt?
2. At 3 tiles/s, how far per frame at 60 fps and at 144 fps?

Goal: Draw shapes and sprite frames, and understand draw order and the camera.

What you will learn

- The canvas origin is the top-left and y grows downward.
- `fillRect`, `arc`, `fillText` and `drawImage` are the main tools.
- `drawImage(sheet, sx, sy, sw, sh, dx, dy, dw, dh)` copies one frame out of a sprite sheet.
- `translate` and `scale` move the camera; `save` / `restore` limit a transform to part of a frame.
- Later draws cover earlier ones, so order is layering.

Find it in your file

- `drawGame`: map, ripples, hiding spots, containers, stairs, doors, player, items, projectiles, enemies, effects, prompts, then the HUD.
- `drawPlayer`: `animationFrame = Math.floor(walkDistance / pxPerFrame) % frameCount` (animation driven by distance, not time).
- `getCamera`, `drawBar`, `drawEnemies`.

Camera and layering

```
context.save();
context.translate(Math.floor(DISPLAY_WIDTH / 2), Math.floor(DISPLAY_HEIGHT / 2));
context.translate(-Math.round(camera.x), -Math.round(camera.y));
drawMap(); /* ... */ drawPlayer(timestamp); drawEnemies();
context.restore(); // the camera ends here
drawHUD(); // drawn in screen space, fixed on screen
```

Try it

- Change the colour of enemies in the `drawEnemies` body.
- Draw a small circle at `playerFeet()` to see where the player's feet are.
- Move `drawPlayer(timestamp)` above `drawMap()` and describe what you see. Undo it.

Check yourself

1. Why is the HUD drawn after `context.restore()`?
2. Why is the map drawn first?

LESSON 9

Arrays and objects in depth

60 min

Goal: Use array methods and modern syntax confidently.

What you will learn

- `find` returns the first match; `filter` returns all matches; `some` / `every` return true or false; `map` transforms; `reduce` folds a list into one value.
- `push`, `splice`, `indexOf`, `includes` change or search arrays.
- Destructuring `const [a, b] = pair`; spread `[...a, ...b]` and `{...obj}`.
- `?.` stops at null/undefined; `??` supplies a default for null/undefined only; `??=` assigns only if missing.

Find it in your file

- `getNearbyHidingSpot` (`find`), `isPlayerInCoreCone` used with `filter`.
- `scrollCount` (`reduce`), `getInventorySlots` (`Object.entries`).
- `spot.suspects ??= new Map()` in `noteSuspicionAtSpot`.
- `[...doors, ...brokenDoors]` in `createGameState`.

Array methods from your file

```
const nearby = hidingSpots.find((s) => !s.occupied && distanceToHidingSpot(s) <= HIDE_INTERACT_DIST);
const witnesses = gameState.enemies.filter(isPlayerInCoreCone);
const total = (gameState.inventory[type] || []).reduce((a, b) => a + b, 0);
spot.suspects ??= new Map(); // only create it if it does not exist yet
```

Try it

- Write `enemiesNotPatrolling()` that returns enemies whose `state` is not `"patrol"`.
- Log how many enemies each new dungeon has using `filter`.
- Replace a `for` loop that searches for something with `find`.

Check yourself

1. Difference between `find` and `filter`?
2. How does `??` differ from `||`?

LESSON 10

Collision and movement

60 min

Goal: See how a hitbox, tile checks and axis-by-axis movement make free movement feel good.

What you will learn

- A hitbox is a small rectangle inside the sprite used for collisions.
- Check the four corners of the hitbox against solid tiles.
- Moving x and y separately lets the player slide along walls.
- Distances use `Math.hypot(dx, dy)`.

Find it in your file

- `HITBOX`, `ENEMY_HITBOX`, `isBoxClear`, `tryMove`, `canMoveTo`.
- `wouldStackOnEnemy` and `separateBodies` (bodies pushing apart).
- `knockback` calling `tryMove` in small steps.

Sliding movement

```
if (isBoxClear(entity.x + dx, entity.y, hb, avoidTraps)) entity.x += dx;
if (isBoxClear(entity.x, entity.y + dy, hb, avoidTraps)) entity.y += dy;
```

Debug view of the player hitbox (temporary, end of drawPlayer)

```
context.strokeStyle = "lime";
context.strokeRect(
  (p.x + HITBOX.left) * TILE_SIZE, (p.y + HITBOX.top) * TILE_SIZE,
  (HITBOX.right - HITBOX.left) * TILE_SIZE, (HITBOX.bottom - HITBOX.top) * TILE_SIZE
);
```

Try it

- Show the hitbox, then change `HITBOX.bottom` and see how close to walls you can stand.
- Walk diagonally into a wall corner and note the sliding.
- Try combining both axes into one `if` in a copy and describe the difference.

Check yourself

1. Why test x and y separately?
2. Why does `HITBOX` start at `top: 0.5`?

LESSON 11

AI state machines and pathfinding

90 min

Goal: Understand how enemies decide what to do and how they find a route.

What you will learn

- A state machine has one current state and clear rules for changing it.
- Suspicion is a number that fills and decays; thresholds trigger transitions.
- Vision uses distance, angle (`angleDiff`) and line of sight (`castRay`).
- Breadth-first search (BFS) explores a grid in rings from the start, so the first time it reaches the goal is the shortest path.

Find it in your file

- `updateEnemyAI`, `alertEnemy`, `startInvestigating`, `updateSearch`.
- `hasLineOfSight`, `castRay`, `angleDiff`, `turnToward`.
- `findPath` (queue plus `prev` map) and `planSteer` / `steerToward`.

The state flow in your game

```
patrol -> curious : noise heard, or suspicion >= CURIOUS_THRESHOLD (0.4)
curious -> alert : suspicion reaches 1, or player in the inner cone
alert -> search : arrived at last seen position without seeing the player
alert -> curious : player lost for ALERT_LOSE_TIME (3 s)
curious -> patrol : finished waiting at the investigation point
```

BFS core

```
const queue = [[sx, sy]];
for (let head = 0; head < queue.length; head++) {
  const [x, y] = queue[head];
  for (const [dx, dy] of dirs) {
    /* skip walls and visited tiles */
    prev.set(k, key(x, y)); // remember where we came from
    queue.push([nx, ny]);
  }
}
```

Try it

- Uncomment `drawVisionCones()` in `drawGame` and watch the cones as you walk.
- Add `console.log(e.id, e.state)` where states change.
- Change `ENEMY_OUTER_RANGE` and `SUSPICION_FILL_FAR` and describe the effect on difficulty.

Check yourself

1. Why does BFS give the shortest path on this grid?
2. What is the `prev` map for?

LESSON 12

Data-driven design and generation

90 min

Goal: Add content by editing data, and understand how a random dungeon is built.

What you will learn

- Data-driven design: behaviour is read from tables such as `CONTAINER_TYPES`, so new content needs no new logic.
- Weighted random choice: weights are relative chances.
- Procedural generation: build randomly, validate, and retry on failure.
- Separate generating the world from drawing it.

Find it in your file

- `CONTAINER_TYPES`, `pickWeighted`, `makeContainer`.
- `createGameState`: `attachRoom`, `rectsOverlap`, carving rooms and connectors, door placement.
- `createDungeon` retrying up to 20 times; `rollDownPlans` for stairs.

Add a container type by adding data only

```
urn: {
  label: "Urn", placement: "center", color: "#57534e", searchMs: 1000, spawnWeight: 1,
  loot: [{ weight: 70, items: [] }, { weight: 30, items: [{ id: "healthPotion", min: 1, max: 1 }] }],
  emptyMessages: ["Only ashes."],
},
```

Try it

- Add the urn above to `CONTAINER_TYPES` and find one in play.
- Set `ROOM_COUNT` to 20, then set `WIDE_CONNECTOR_CHANCE` to 0.6. Describe the layouts.
- Count how many times `createDungeon` retries by logging inside its loop.

Check yourself

1. Why does `createDungeon` try up to 20 times?
2. In a loot table with weights 25, 35, 25, 15, what is the chance of the second row?

Part 2 - Adding stealth features

Design brief

Each feature you asked for is mapped to a lesson and to the parts of your file it touches. The code is given as small additions and edits to your existing functions, not as replacement files, so your own changes stay intact. Where the plan says BEFORE / AFTER, change only those lines. Interpretation used in Lesson 16: only torch-bearing enemies leave their rooms to patrol the whole dungeon; torchless enemies keep their posts.

Feature	Lesson	Hooks into
Map torches that can be lit and put out	13	<code>createGameState</code> , <code>KeyE</code> handler, <code>drawGame</code>
Torches carried by player and enemies	13, 16	<code>getLightSources</code> , enemy objects
Throw torches (attack or distraction)	15	<code>throwItem</code> , <code>landThrown</code>
Throwable water that extinguishes torches	15	<code>landThrown</code>
Hide in shadows when crouched	14	<code>isPlayerInLight</code> (your stub), <code>updateEnemyAI</code>
Throw stones and other distractions	15	<code>makeNoise</code> , <code>startInvestigating</code>
Dungeon-wide patrols, torch enemies only	16	<code>spawnEnemies</code> , <code>planSteer</code> , patrol branch
Modular rooms with multiple paths	17	<code>createGameState</code> connectors
Cover: pillars, crates, low walls	17	<code>TileType</code> , <code>placeCover</code> , <code>drawMap</code>

LESSON 13

Torches and lighting

90 min

Goal: Add wall torches you can light and extinguish, carried torches, and a darkness layer that only light can erase.

Design rules

- Light sources: lit wall torches, lit dropped torches, torches carried by enemies, and the player's torch.
- A point is lit when it is inside a source's radius and `hasLineOfSight` from the source (light does not pass walls or closed doors).
- Putting out a torch near enemies makes them investigate ("who put that out?").
- A lit player torch means the player is always visible: a trade-off between seeing and being seen.

Touches in your file

- New: `placeWallTorches`, `getLightSources`, `isLitAt`, `getNearbyTorch`, `toggleTorch`, `drawLighting`.
- Edit: the `isPlayerInLight` stub; the `KeyE` handler; `createGameState` (player and returned object); `gameLoop`; `drawGame`; `useStairs`.

A. Constants and wall-torch placement (call it after `placeHidingSpots`)

```
const TORCH_RADIUS = 4.5;           // tiles of light
const TORCH_INTERACT_DIST = 1.1;
const TORCHES_PER_ROOM_MAX = 2;
const TORCH_LIT_CHANCE = 0.7;       // wall torches start lit this often
const TORCH_NOTICE_RANGE = 8;        // enemies this close notice a torch going out
const AMBIENT_DARKNESS = 0.7;        // 0 = bright, 1 = pitch black

function placeWallTorches(rooms, map) {
  const torches = [];
  for (const room of rooms) {
    const spots = [];
    for (let x = room.x; x < room.x + room.w; x++) {
      if (map[room.y - 1][x] !== TileType.WALL) continue; // doorway above: skip
      if (hidingSpots.some((h) => Math.hypot(h.x - x, h.y - room.y) < 1.5)) continue;
      spots.push({ x, y: room.y });
    }
  }
}
```

```

    }
    spots.sort(() => Math.random() - 0.5);

    const wanted = rollInt(0, TORCHES_PER_ROOM_MAX);
    let placed = 0;
    for (const s of spots) {
        if (placed >= wanted) break;
        if (torches.some((t) => Math.hypot(t.x - s.x, t.y - s.y) < 3)) continue;
        torches.push({ x: s.x, y: s.y, lit: Math.random() < TORCH_LIT_CHANCE,
            radius: TORCH_RADIUS, kind: "wall" });
        placed++;
    }
    }
    return torches;
}

```

B. Light sources and the lit test (replace the isPlayerInLight stub)

```

function getLightSources() {
    const sources = [];
    for (const t of gameState.torches) if (t.lit) sources.push(t);
    for (const e of gameState.enemies) {
        if (e.torch?.lit) sources.push({ x: e.x, y: e.y, radius: TORCH_RADIUS });
    }
    const p = gameState.player;
    if (p.torch?.lit && !p.hidden) sources.push({ x: p.x, y: p.y, radius: TORCH_RADIUS });
    return sources;
}

// (x, y) is in "centre" coordinates, like playerCenter().
function isLitAt(x, y) {
    for (const s of getLightSources()) {
        const sx = s.x + 0.5;
        const sy = s.y + 0.5;
        if (Math.hypot(sx - x, sy - y) > s.radius) continue;
        if (hasLineOfSight(sx, sy, x, y)) return true;
    }
    return false;
}

function isPlayerInLight() { // REPLACES the stub that returned false
    const pc = playerCenter();
    return isLitAt(pc.x, pc.y);
}

```

C. Interacting: E on a wall torch. T for your own torch

```

function getNearbyTorch() {
    const pc = playerCenter();
    let best = null;
    let bestDist = TORCH_INTERACT_DIST;
    for (const t of gameState.torches) {
        if (t.kind !== "wall") continue;
        const d = Math.hypot(t.x + 0.5 - pc.x, t.y + 0.5 - pc.y);
        if (d <= bestDist) { best = t; bestDist = d; }
    }
    return best;
}

function toggleTorch(t) {
    t.lit = !t.lit;
    if (t.lit) return;
    for (const e of gameState.enemies) { // "who put that out?"
        if (e.state === "alert") continue;
        const near = Math.hypot(e.x - t.x, e.y - t.y) <= TORCH_NOTICE_RANGE;
        if (near && hasLineOfSight(e.x + 0.5, e.y + 0.5, t.x + 0.5, t.y + 0.5)) {
            startInvestigating(e, t.x, t.y);
        }
    }
}

// In the KeyE handler, after "if (startContainerSearch()) return;" and before toggleNearbyDoor():
const torch = getNearbyTorch();
if (torch) { toggleTorch(torch); return; }

// New handler for your own torch:
window.addEventListener("keydown", (event) => {
    if (event.code !== "KeyT" || event.repeat) return;
    if (!gameRunning || !gameState || inventoryOpen) return;
    const t = gameState.player.torch;

```

```

    if (t) t.lit = !t.lit;
  });

```

D. The darkness layer (an offscreen canvas the size of the map)

```

const lightLayer = document.createElement("canvas");
lightLayer.width = MAP_WIDTH * TILE_SIZE;
lightLayer.height = MAP_HEIGHT * TILE_SIZE;
const lightCtx = lightLayer.getContext("2d");

function drawLighting(now) {
  lightCtx.globalCompositeOperation = "source-over";
  lightCtx.clearRect(0, 0, lightLayer.width, lightLayer.height);
  lightCtx.fillStyle = `rgba(0, 0, 10, ${AMBIENT_DARKNESS})`;
  lightCtx.fillRect(0, 0, lightLayer.width, lightLayer.height);

  lightCtx.globalCompositeOperation = "destination-out"; // erase darkness where lit
  for (const s of getLightSources()) {
    const cx = (s.x + 0.5) * TILE_SIZE;
    const cy = (s.y + 0.5) * TILE_SIZE;
    const flicker = 1 + Math.sin(now / 130 + s.x * 7) * 0.04;
    const r = s.radius * TILE_SIZE * flicker;
    const g = lightCtx.createRadialGradient(cx, cy, 4, cx, cy, r);
    g.addColorStop(0, "rgba(0,0,0,1)");
    g.addColorStop(1, "rgba(0,0,0,0)");
    lightCtx.fillStyle = g;
    lightCtx.beginPath();
    lightCtx.arc(cx, cy, r, 0, Math.PI * 2);
    lightCtx.fill();
  }
  lightCtx.globalCompositeOperation = "source-over";
  context.drawImage(lightLayer, 0, 0); // drawn in world space, under the camera
}

function drawTorches(now) {
  for (const t of gameState.torches || []) {
    const px = t.x * TILE_SIZE;
    const py = t.y * TILE_SIZE;
    const bx = px + TILE_SIZE / 2 - 2;
    const by = t.kind === "wall" ? py + 2 : py + TILE_SIZE / 2;
    context.fillStyle = "#78350f";
    context.fillRect(bx, by + 4, 4, 8);
    if (!t.lit) continue;
    const h = 5 + Math.sin(now / 90 + t.x) * 2;
    context.fillStyle = "#fb923c";
    context.fillRect(bx - 1, by + 4 - h, 6, h);
    context.fillStyle = "#fde047";
    context.fillRect(bx, by + 6 - h, 4, h - 2);
  }
}

```

E. Wiring (add each line next to the one named)

```

// createGameState, new-player object:      torch: { lit: false },
// createGameState, returned object:        torches: placeWallTorches(rooms, newMap),
//                                           thrown: [],
//                                           ammo: { stone: 5, bottle: 1, water: 2 },
//                                           selectedThrow: "stone",
// useStairs, with the other "target." lines:
target.ammo = from.ammo;
target.selectedThrow = from.selectedThrow;
from.thrown.length = 0;
// drawGame, after drawContainers():        drawTorches(timestamp);
// drawGame, after drawEffects(timestamp, "air"): drawLighting(timestamp);

```

Test it

- Light and put out a wall torch with E. The room should brighten and darken.
- Press T to toggle your torch. Confirm enemies in view of it can see you more easily (Lesson 14).
- Lower **AMBIENT_DARKNESS** to 0.5 if the view is too dark on your 8x8 tile screen.
- Stretch: clip each light to walls by reusing the flood fill from **soundReach** (the way **drawRipples** clips).

Check yourself

1. Why does **isLitAt** call **hasLineOfSight**?
2. What does "destination-out" do?

Goal: Let a crouched player hide in shadow and behind low cover.

Design rules

- Crouching is your existing "sneak" gait (key C).
- Crouched and not lit: enemies can only notice you within `SHADOW_SIGHT_RANGE`.
- Your file already had the hook: `isPlayerExposed` uses `isPlayerInLight()`, so standing in light now makes you exposed even when still.
- A low wall does not block sight for a standing player, but hides a crouched player from enemies that are not already alert.

Touches in your file

- New: `isCrouchedInShadow`, `isCoveredByLowWall`.
- Edit: two spots inside `updateEnemyAI`.

A. New helpers

```
const SHADOW_SIGHT_RANGE = 1.3;    // crouched in shadow: noticed only this close

function isCrouchedInShadow() {
  const p = gameState.player;
  return p.gait === "sneak" && !p.hidden && !isPlayerInLight();
}

function isCoveredByLowWall(ax, ay, bx, by) {
  if (gameState.player.gait !== "sneak") return false;
  const steps = Math.ceil(Math.hypot(bx - ax, by - ay) / 0.25);
  for (let i = 1; i < steps; i++) {
    const t = i / steps;
    const tx = Math.floor(ax + (bx - ax) * t);
    const ty = Math.floor(ay + (by - ay) * t);
    if (gameState.map[ty]?.[tx] === TileType.LOWWALL) return true;
  }
  return false;
}
```

B. Edit `updateEnemyAI` (change only these lines)

```
// BEFORE
const sightRange =
  e.state === "alert" ? ALERT_SIGHT_RANGE : ENEMY_OUTER_RANGE;

// AFTER
let sightRange = e.state === "alert" ? ALERT_SIGHT_RANGE : ENEMY_OUTER_RANGE;
if (e.state !== "alert" && isCrouchedInShadow()) {
  sightRange = Math.min(sightRange, SHADOW_SIGHT_RANGE);
}

// The sight test: add ONE condition (LOWWALL is created in Lesson 17)
if (
  !playerHidden &&
  dist <= sightRange &&
  hasLineOfSight(ec.x, ec.y, pc.x, pc.y) &&
  (e.state === "alert" || !isCoveredByLowWall(ec.x, ec.y, pc.x, pc.y)) // NEW
) {
```

Situation	Result
Standing in torch light	Exposed: normal cones and suspicion apply
Moving in the dark, not crouched	Exposed while moving (your existing rule)
Crouched in shadow	Noticed only within 1.3 tiles
Crouched behind a low wall	Not seen by non-alert enemies
Carrying a lit torch	Always lit, so never counts as shadow

Test it

- Stand still and crouched in a dark corner while a torch enemy walks past at 3 or more tiles: you should not be noticed.
- Press T (lit torch) in the same spot: the enemy should now notice you.
- Stretch: add a small "SHADOW / LIT" indicator to `drawHUD` that shows `isCrouchedInShadow()`.
- Performance note: `isCrouchedInShadow()` runs once per enemy per frame. Cache it once per frame in `updateEnemies` when you are ready.

Check yourself

1. Why check `e.state !== "alert"` before shrinking sight range?
2. Why does a low wall hide only a crouched player?

LESSON 15

Throwing: stones, torches and water

90 min

Goal: Add throwable objects that make noise, hurt, light or put out fires.

Design rules

- R cycles the selected throwable; F throws it toward the mouse. It lands where you aim, up to `THROW_MAX_DIST`.
- Every landing calls your existing `makeNoise`, so distractions reuse your enemy-hearing system.
- Stone and bottle: a distraction (the bottle is loud). Torch: damage, plus a burning torch on the ground for a while. Water: puts out torches within a radius.
- Enemies can throw torches too (Lesson 16), using the same `thrown` list with `thrower: "enemy"`.

Touches in your file

- New: `THROWABLES`, `throwItem`, `updateThrown`, `landThrown`, `drawThrown`, `updateTorches`.
- Edit: `gameLoop` (two update calls), `drawGame` (`drawThrown`), `drawHUD` (selected item).

A. Data and throwing

```
const THROW_SPEED = 9;           // tiles per second
const THROW_MAX_DIST = 7;
const STONE_HIT_DAMAGE = 5;
const TORCH_HIT_DAMAGE = 20;
const TORCH_BURN_MS = 12000;     // a dropped torch burns this long
const WATER_SPLASH_RADIUS = 2;
let lastThrownTime = null;

const THROWABLES = {
  stone: { label: "Stone",      noise: 6, color: "#a1a1aa" },
  bottle: { label: "Bottle",    noise: 8, color: "#86efac" }, // glass: loud
  torch: { label: "Torch",     noise: 2, color: "#fb923c" },
  water: { label: "Water flask", noise: 3, color: "#38bdf8" },
};
const THROW_KINDS = Object.keys(THROWABLES);

function throwItem(kind, dx, dy) {
  const p = gameState.player;
  if (kind === "torch") {
    if (!p.torch) return false;
    p.torch = null; // the torch leaves your hand
  } else {
    if (!(gameState.ammo[kind] > 0)) return false;
    gameState.ammo[kind]--;
  }
  const len = Math.hypot(dx, dy) || 1;
  gameState.thrown.push({
    kind, thrower: "player",
    x: p.x, y: p.y + 0.2, dx: dx / len, dy: dy / len,
    travelled: 0, maxDist: Math.min(THROW_MAX_DIST, len),
  });
  return true;
}
```

```

window.addEventListener("keydown", (event) => {
  if (!gameRunning || !gameState || event.repeat || inventoryOpen) return;
  if (event.code === "KeyR") {
    const i = THROW_KINDS.indexOf(gameState.selectedThrow);
    gameState.selectedThrow = THROW_KINDS[(i + 1) % THROW_KINDS.length];
  }
  if (event.code === "KeyF" && !gameState.player.hidden) {
    const aim = getAimVector();
    if (aim) throwItem(gameState.selectedThrow, aim.dx, aim.dy);
  }
});

```

B. Flight and landing

```

function updateThrown(timestamp) {
  const dt = lastThrownTime === null ? 0 : Math.min((timestamp - lastThrownTime) / 1000, 0.05);
  lastThrownTime = timestamp;
  if (!gameState || !gameState.thrown) return;
  const p = gameState.player;

  for (let i = gameState.thrown.length - 1; i >= 0; i--) {
    const t = gameState.thrown[i];
    const step = Math.min(THROW_SPEED * dt, t.maxDist - t.travelled);
    t.x += t.dx * step;
    t.y += t.dy * step;
    t.travelled += step;

    let victim = null;
    if (t.thrower === "player") {
      victim = gameState.enemies.find(
        (e) => Math.hypot(e.x - t.x, e.y - t.y) < PROJECTILE_HIT_RADIUS) || null;
    } else if (!p.hidden && Math.hypot(p.x - t.x, p.y - t.y) < 0.6) {
      victim = p;
    }
    const hitWall = isWallAt(t.x + 0.5, t.y + 0.5);
    if (hitWall) { t.x -= t.dx * step; t.y -= t.dy * step; }

    if (victim || hitWall || t.travelled >= t.maxDist) {
      landThrown(t, victim, timestamp);
      gameState.thrown.splice(i, 1);
    }
  }
}

function landThrown(t, victim, now) {
  const p = gameState.player;
  const def = THROWABLES[t.kind];
  const near = (obj, r) => Math.hypot(obj.x - t.x, obj.y - t.y) <= r;

  if (t.kind === "stone" || t.kind === "bottle") {
    if (victim && victim !== p) hurtEnemy(victim, STONE_HIT_DAMAGE, now);
  } else if (t.kind === "torch") {
    gameState.torches.push({ x: t.x, y: t.y, lit: true, radius: TORCH_RADIUS,
      kind: "dropped", burnUntil: now + TORCH_BURN_MS });
    if (victim === p) damagePlayer(TORCH_HIT_DAMAGE);
    else if (victim) hurtEnemy(victim, TORCH_HIT_DAMAGE, now);
  } else if (t.kind === "water") {
    for (const s of gameState.torches) if (near(s, WATER_SPLASH_RADIUS)) s.lit = false;
    for (const e of gameState.enemies) {
      if (!e.torch?.lit || !near(e, WATER_SPLASH_RADIUS)) continue;
      e.torch.lit = false;
      if (e.state !== "alert") startInvestigating(e, t.x, t.y);
    }
    if (p.torch?.lit && near(p, WATER_SPLASH_RADIUS)) p.torch.lit = false;
    (gameState.effects || []).push({ kind: "burst", x: t.x, y: t.y,
      radius: WATER_SPLASH_RADIUS, color: "#38bdf8", start: now, until: now + 350 });
  }
  makeNoise(t.x, t.y, def.noise, def.color); // every landing is a sound
}

function updateTorches(timestamp) {
  if (!gameState || !gameState.torches) return;
  gameState.torches = gameState.torches.filter(
    (t) => !(t.kind === "dropped" && timestamp > t.burnUntil));
}

function drawThrown() {
  for (const t of gameState.thrown || []) {
    context.fillStyle = THROWABLES[t.kind].color;
    context.beginPath();
  }
}

```

```

    context.arc((t.x + 0.5) * TILE_SIZE, (t.y + 0.5) * TILE_SIZE, 3, 0, Math.PI * 2);
    context.fill();
  }
}

```

C. Wiring and HUD line

```

// gameLoop, after updateProjectiles(timestamp);
updateThrown(timestamp);
updateTorches(timestamp);
// drawGame, after drawProjectiles();
drawThrown();
// drawHUD, after the armed-scroll text:
const k = gameState.selectedThrow;
const n = k === "torch" ? (p.torch ? 1 : 0) : gameState.ammo[k];
context.fillStyle = "#e2e8f0";
context.fillText(`[R] ${THROWABLES[k].label} x${n}`, x, 44);

```

Test it

- Throw a stone past an enemy: nearby non-alert enemies should walk to where it landed (look for the ripple).
- Throw water on a wall torch, then on a torch-carrying enemy (Lesson 16).
- Throw a lit torch at an enemy: damage, alert, and a burning torch lighting the floor for about 12 seconds.
- Stretch: let the player pick up a dropped torch with E, and add more "other items" by adding rows to **THROWABLES**.

Check yourself

1. Why does **landThrown** always call **makeNoise**?
2. Why is **maxDist** the smaller of **THROW_MAX_DIST** and the aim length?

LESSON 16

90 min

Torch patrols across the dungeon

Goal: Give some enemies a torch and a route through many rooms, including closed doors.

Design rules

- Torch-bearing patrollers are a few enemies upgraded after normal spawning (**PATROLLERS_BASE** plus one every two depths).
- A patroller's route is 3-5 rooms; it opens doors in its way.
- Torches are the weakness: put one out (water) or lose it (thrown) and the enemy walks back to its old post (**homePatrol**).
- A patroller can throw its torch at a visible player, which lights the area where it lands.

Touches in your file

- New: **makeDungeonPatrollers**, **floorSpotIn**, **openDoorsNear**.
- Edit: **buildEnemy** (two fields), **planSteer** (one argument), the patrol branch of **updateEnemyAI**, the alert branch, and **createGameState**.

A. Unrading enemies into patrollers

```

const PATROLLERS_BASE = 1;
const PATROLLER_EVERY_N_LEVELS = 2;
const PATROL_ROOMS_MIN = 3;
const PATROL_ROOMS_MAX = 5;
const ENEMY_THROW_INTERVAL_MS = 6000;

function floorSpotIn(room, map) {
  const tiles = [];
  for (let y = room.y; y < room.y + room.h; y++)
    for (let x = room.x; x < room.x + room.w; x++)
      if (map[y][x] === TileType.FLOOR) tiles.push({ x, y });
  return pickOne(tiles);
}

function makeDungeonPatrollers(enemies, rooms, map, level) {
  const count = Math.min(
    enemies.length,

```

```

    PATROLLERS_BASE + Math.floor((level - 1) / PATROLLER_EVERY_N_LEVELS)
  );
  const chosen = enemies.slice().sort(() => Math.random() - 0.5).slice(0, count);

  for (const e of chosen) {
    const stops = rooms.slice(1).sort(() => Math.random() - 0.5)
      .slice(0, rollInt(PATROL_ROOMS_MIN, PATROL_ROOMS_MAX));
    const route = stops.map((r) => floorSpotIn(r, map)).filter(Boolean)
      .map((t) => ({ x: t.x, y: t.y, wait: spawnRand(800, 2000) }));
    if (route.length < 2) continue;

    e.homePatrol = e.patrol;          // its old post, used when its torch is out
    e.patrol = route;
    e.patrolIndex = 0;
    e.role = "patroller";
    e.torch = { lit: true };
    e.holdFacing = null;
    e.waitTimer = 0;
  }
}

function openDoorsNear(e) {
  for (const d of gameState.doors) {
    if (!d.open && Math.hypot(d.x - e.x, d.y - e.y) < 1.2) d.open = true;
  }
}

```

B. Edits to existing functions

```

// buildEnemy: add two fields to the returned object
torch: null,
homePatrol: null,

// createGameState: right after "const enemies = spawnEnemies(...)"
makeDungeonPatrollers(enemies, rooms, newMap, level);

// planSteer: add a 6th argument to the findPath call
avoid,
e.role === "patroller"          // may plan a route through closed doors
);

// updateEnemyAI, patrol branch (the final else): BEFORE
const wp = e.patrol[e.patrolIndex];
e.patrolIndex = (e.patrolIndex + 1) % e.patrol.length;
// AFTER (add the first two lines above "if (e.waitTimer > 0)")
if (e.role === "patroller") openDoorsNear(e);
const route = e.homePatrol && !e.torch?.lit ? e.homePatrol : e.patrol;
const wp = route[e.patrolIndex % route.length];
e.patrolIndex = (e.patrolIndex + 1) % route.length;

```

C. Enemy throws its torch (in the alert branch, after the melee check)

```

if (
  e.canSeePlayer && e.torch?.lit &&
  dist > ENEMY_MELEE_REACH + 1 && dist < 6 &&
  now >= (e.nextThrowTime ?? 0)
) {
  e.nextThrowTime = now + ENEMY_THROW_INTERVAL_MS;
  e.torch = null;                                // the torch leaves its hand
  gameState.thrown.push({
    kind: "torch", thrower: "enemy",
    x: e.x, y: e.y + 0.2,
    dx: Math.cos(angleToPlayer), dy: Math.sin(angleToPlayer),
    travelled: 0, maxDist: dist,
  });
}

```

Test it

- Watch a patroller: it should walk through at least three rooms and open a door on the way.
- Soak its torch with water: it should investigate, then return to its old post.
- Let one spot you in the open: it should throw its torch once, then walk home.
- Stretch: let torchless enemies relight at the nearest wall torch; make patrollers close doors behind them.

Check yourself

1. Why is `passDoors` limited to patrollers?
2. What stops a torchless enemy from roaming the dungeon?

LESSON 17

Modular rooms, loops and cover

120 min

Goal: Make every dungeon offer more than one route between rooms, and put cover in every room.

Design rules

- Your generator grows a tree: every room hangs off a parent, so there is exactly one path between any two rooms. Extra connectors between rooms that already sit close together create loops (multiple paths).
- Cover comes from small reusable "chunks" written as text grids, stamped into room interiors (never on the wall ring, doors, stairs, traps, containers or hiding spots).
- After each stamp a flood fill checks that all walkable tiles are still connected; if not, the stamp is undone.
- Pillars and crates block movement and sight; low walls block movement only (Lesson 14 uses them to hide crouching players).

Touches in your file

- New: `addLoopConnectors`, `COVER_MODULES`, `placeCover`, `floodCount`, `blocksSight`.
- Edit: `TileType`, `isSolidTile`, `isWalkableTile`, `isWallAt`, `drawMap` (duplicate FLOOR block), and `createGameState`.

A. Tiles that block movement and sight

```
const TileType = { WALL: 0, FLOOR: 1, DOOR: 2, TRAP: 3, SWITCH: 4, PILLAR: 5,
                  CRATE: 6, LOWWALL: 7 };          // CRATE/LOWWALL are new

function isSolidTile(t) {
  return t === TileType.WALL || t === TileType.PILLAR ||
         t === TileType.CRATE || t === TileType.LOWWALL;
}
function blocksSight(t) {
  return t === TileType.WALL || t === TileType.PILLAR || t === TileType.CRATE;
}

// isWalkableTile: last line becomes
return !isSolidTile(gameState.map[tileY][tileX]);
// isWallAt: the wall check becomes
if (blocksSight(gameState.map[ty][tx])) return true;
```

B. Loop connectors (inside `createGameState` after the room-placement while loop)

```
// In the while loop, store which rooms a connector joins:
connectors.push({ ...attached, a: parent, b: room });

function addLoopConnectors(maxGap, wanted) {
  const linked = (a, b) =>
    connectors.some(c => (c.a === a && c.b === b) || (c.a === b && c.b === a));
  let added = 0;
  for (let i = 0; i < rooms.length && added < wanted; i++) {
    for (let j = i + 1; j < rooms.length && added < wanted; j++) {
      const a = rooms[i];
      const b = rooms[j];
      if (linked(a, b)) continue;
      const overlapY = Math.min(a.y + a.h, b.y + b.h) - Math.max(a.y, b.y);
      const overlapX = Math.min(a.x + a.w, b.x + b.w) - Math.max(a.x, b.x);
      let rect = null;
      let horizontal = false;

      if (overlapY >= 1) {
        const [L, R] = a.x < b.x ? [a, b] : [b, a];
        const gap = R.x - (L.x + L.w);
        if (gap >= 1 && gap <= maxGap) {
          const lane = randInt(Math.max(a.y, b.y), Math.min(a.y + a.h, b.y + b.h) - 1);
          rect = { x: L.x + L.w, y: lane, w: gap, h: 1 };
          horizontal = true;
        }
      }
    }
  }
}
```

```

    }
  } else if (overlapX >= 1) {
    const [T, B] = a.y < b.y ? [a, b] : [b, a];
    const gap = B.y - (T.y + T.h);
    if (gap >= 1 && gap <= maxGap) {
      const lane = randint(Math.max(a.x, b.x), Math.min(a.x + a.w, b.x + b.w) - 1);
      rect = { x: lane, y: T.y + T.h, w: 1, h: gap };
    }
  }
  if (!rect) continue;
  if (rooms.some((r) => r !== a && r !== b && rectsOverlap(rect, r, 1))) continue;
  if (connectors.some((c) => rectsOverlap(rect, c.rect, 1))) continue;
  connectors.push({ rect, wide: false, horizontal, a, b });
  added++;
}
}
return added;
}

const MIN_LOOPS = 2;
if (addLoopConnectors(4, 3) < MIN_LOOPS) return null; // createDungeon retries

```

C. Cover modules, stamping and the connectivity check

```

const COVER_LEGEND = { P: TileType.PILLAR, C: TileType.CRATE, L: TileType.LOWWALL };
const COVER_MODULES = [
  { name: "twin pillars", rows: ["P.P"] },
  { name: "crate stack", rows: ["CC", "C."] },
  { name: "low wall", rows: ["LLL"] },
  { name: "L barricade", rows: ["LL", "L."] },
  { name: "pillar & crate", rows: ["P.", ".C"] },
  { name: "single pillar", rows: ["P"] },
];

function floodCount(map, sx, sy) {
  const seen = new Set([sy * MAP_WIDTH + sx]);
  const queue = [[sx, sy]];
  for (let head = 0; head < queue.length; head++) {
    const [x, y] = queue[head];
    for (const [dx, dy] of [[1, 0], [-1, 0], [0, 1], [0, -1]]) {
      const nx = x + dx;
      const ny = y + dy;
      if (nx < 0 || ny < 0 || nx >= MAP_WIDTH || ny >= MAP_HEIGHT) continue;
      if (isSolidTile(map[ny][nx])) continue;
      const k = ny * MAP_WIDTH + nx;
      if (seen.has(k)) continue;
      seen.add(k);
      queue.push([nx, ny]);
    }
  }
  return seen.size;
}

function countWalkable(map) {
  let n = 0;
  for (const row of map) for (const t of row) if (!isSolidTile(t)) n++;
  return n;
}

// reserved: Set of "x,y" strings (traps, stairs, containers, hiding spots, doors, spawn)
function placeCover(rooms, map, reserved) {
  const start = rooms[0];
  for (const room of rooms) {
    const wanted = room.w * room.h >= 30 ? 2 : 1;
    let placed = 0;
    const mods = COVER_MODULES.slice().sort(() => Math.random() - 0.5);
    mods.push(COVER_MODULES[COVER_MODULES.length - 1]); // guaranteed fallback

    for (const mod of mods) {
      if (placed >= wanted) break;
      const mh = mod.rows.length;
      const mw = mod.rows[0].length;
      const spots = [];
      for (let y = room.y + 1; y + mh <= room.y + room.h - 1; y++)
        for (let x = room.x + 1; x + mw <= room.x + room.w - 1; x++) spots.push({ x, y });
      spots.sort(() => Math.random() - 0.5);

      for (const s of spots) {
        const cells = [];
        let ok = true;

```

```

    for (let r = 0; r < mh && ok; r++) {
      for (let c = 0; c < mw; c++) {
        const ch = mod.rows[r][c];
        if (ch === ".") continue;
        const tx = s.x + c;
        const ty = s.y + r;
        if (map[ty][tx] !== TileType.FLOOR || reserved.has(`${tx},${ty}`)) { ok = false; break; }
        cells.push({ tx, ty, tile: COVER_LEGEND[ch] });
      }
    }
    if (!ok) continue;
    for (const c of cells) map[c.ty][c.tx] = c.tile;
    const before = countWalkable(map); // after stamping
    if (floodCount(map, start.x + 1, start.y + 1) === before) { placed++; break; }
    for (const c of cells) map[c.ty][c.tx] = TileType.FLOOR; // undo: it cut the level
  }
}
}
}

```

D. Wiring and drawing

```

// createGameState: after placeContainers(...) and before spawnEnemies(...)
const reserved = new Set();
for (const o of [...items, ...stairs, ...containers, ...hidingSpots]) {
  reserved.add(`${o.x},${o.y}`);
}
for (const d of [...doors, ...brokenDoors]) reserved.add(`${d.x},${d.y}`);
reserved.add(`${startRoom.x + 1},${startRoom.y + 1}`); // player spawn
placeCover(rooms, newMap, reserved);

// drawMap: REPLACE the duplicate "tile === FLOOR || tile === PILLAR" block with
if (tile === TileType.PILLAR || tile === TileType.CRATE || tile === TileType.LOWWALL) {
  if (art) context.drawImage(art.floor, tx, ty);
  if (tile === TileType.PILLAR) drawPillar({ x, y });
  else if (tile === TileType.CRATE) {
    context.fillStyle = "#000"; context.fillRect(tx + 3, ty + 5, TILE_SIZE - 6, TILE_SIZE - 8);
    context.fillStyle = "#854d0e"; context.fillRect(tx + 4, ty + 6, TILE_SIZE - 8, TILE_SIZE - 10);
  } else {
    context.fillStyle = "#52525b"; context.fillRect(tx, ty + TILE_SIZE / 2 - 4, TILE_SIZE, 10);
  }
  continue;
}

```

Test it

- Log `connectors.length - (rooms.length - 1)` (the number of loops) and confirm it is at least 2 in every dungeon.
- Walk every room: each should contain at least one pillar, crate or low wall, and you should always be able to walk around it.
- Generation stress test: add the function below, call it from the console on a fresh page (before `startGame3`), then reload. It changes shared arrays such as `hidingSpots`, so never run it during a game.

Stress test (run before starting a game. then reload the page)

```

window.stressTest = (n = 50) => {
  let failed = 0;
  for (let i = 0; i < n; i++) if (!createGameState(1, null, {})) failed++;
  console.log(`${failed}/${n} generations returned null`);
};

```

Try it

- If many generations fail, raise `maxGap` or lower `MIN_LOOPS`. Stretch: add new modules (a 3x3 "ring of pillars"), and a check that removing any one door still leaves every room reachable.

Check yourself

1. Why do you need extra connectors to get multiple paths?
2. Why flood fill after stamping cover?

Part 3 - Putting it together

Integration checklist

#	Check	Expected
1	Light and extinguish a wall torch with E	Room brightens and darkens
2	Crouch (C) in shadow, enemy 3+ tiles away	Not noticed
3	Press T with a lit torch while crouched	Noticed (you are lit)
4	R to pick a stone, F to throw it	Nearby enemies investigate the landing spot
5	Throw water on a torch enemy	Torch out, enemy investigates, returns to post
6	Throw a lit torch at an enemy	20 damage, alert, torch burns on the floor
7	Take stairs down and up	Ammo and selection carried; thrown list cleared
8	Generate 20 dungeons	At least 2 loops each, cover in every room
9	Watch a patroller for one minute	Visits 3+ rooms, opens doors

Debugging habits

Read the red console error from the top: the first line names the problem and the file and line. Use `console.log` to print values, or the DevTools **Sources** tab to set a breakpoint (click a line number) and pause the game. Change one thing at a time. When you add a feature, test it before adding the next.

Capstone ideas

Choose one: (1) design a new enemy type, "sentry", that never moves but has a long narrow vision cone; (2) design three new cover modules and a rule for where each may appear; (3) add torch fuel that drains and can be refilled; (4) make torch light respect walls using the `soundReach` flood fill; (5) write your own Lesson 18 explaining a feature you added.

Appendices

Appendix A - Bug hunt in your current file

These are small real issues in `Lesson_plan.js`. Fixing them is good practice.

Where	Problem	Fix
<code>drawContainerText</code>	<code>context.textBadeline</code> is a typo, so the baseline is never set	<code>textBaseline</code>
<code>drawMap</code>	The second <code>tile === FLOOR PILLAR</code> block: the FLOOR half can never run	Keep only PILLAR there (Lesson 17 replaces it)
<code>drawCorner(art, type, ...)</code>	<code>art</code> and <code>type</code> are never used; inner and outer corners look the same	Choose a sprite by <code>type</code>
<code>drawPillars</code>	Reads <code>gameState.pillars</code> , which is never created	Remove it, or fill <code>pillars</code> in generation
<code>blur</code> listeners	Registered three times	Merge into one handler
<code>lastSeparationTime</code>	Starts at 20000 but the code tests for <code>null</code> like the other timers	Initialise to <code>null</code>
<code>startGame3</code>	Pushes a "debug-scroll" item every run	Remove before release
<code>drawVisionCones()</code>	Commented out with a note to hide it in the final version	Keep off for release

Appendix B - Function map of your file

Section	Functions
Setup and tiles	<code>isSolidTile</code> , <code>getTileArt</code> , <code>sliceTileset</code> , <code>isFloorAt</code> , <code>drawWallEdges</code> , <code>drawMap</code>
Stairs	<code>rollDownPlans</code> , <code>createDungeon</code> , <code>useStairs</code> , <code>updateStairs</code> , <code>drawStairs</code>
Input	<code>keydown</code> and <code>keyup</code> handlers, <code>getMouseWorldTile</code> , <code>getAimVector</code> , <code>directionFromVector</code>
Loop	<code>startGame3</code> , <code>stopGame3</code> , <code>gameLoop</code> , <code>getCamera</code> , <code>revealMinimapView</code>
Player	<code>updateFreePlayerMovement</code> , <code>drawPlayer</code> , <code>playerFeet</code> , <code>playerCenter</code>
Collision	<code>isWalkableTile</code> , <code>isBoxClear</code> , <code>isSegmentClear</code> , <code>tryMove</code> , <code>separateBodies</code> , <code>knockback</code>
Noise	<code>makeNoise</code> , <code>soundReach</code> , <code>spawnRipple</code> , <code>drawRipples</code>
Enemy AI	<code>updateEnemyAI</code> , <code>alertEnemy</code> , <code>startInvestigating</code> , <code>beginSearch</code> , <code>updateSearch</code> , <code>findPath</code> , <code>steerToward</code>
Vision	<code>castRay</code> , <code>hasLineOfSight</code> , <code>angleDiff</code> , <code>drawVisionCones</code>
Hiding	<code>placeHidingSpots</code> , <code>toggleHide</code> , <code>breakHidingSpot</code> , <code>drawHidingSpots</code>
Doors	<code>toggleNearbyDoor</code> , <code>hitDoor</code> , <code>breakDoor</code> , <code>isBoxBlockedByDoor</code> , <code>drawDoors</code>
Containers	<code>placeContainers</code> , <code>makeContainer</code> , <code>startContainerSearch</code> , <code>finishContainerSearch</code>
Inventory	<code>addItemToInventory</code> , <code>drawInventory</code> , <code>useSelectedItem</code> , <code>dropSelectedItem</code>
Spells	<code>castSpell</code> , <code>detonateScroll</code> , <code>chainLightning</code> , <code>updateProjectiles</code> , <code>updateEffects</code>
Generation	<code>createGameState</code> , <code>spawnEnemies</code> , <code>placeContainers</code> , <code>placeHidingSpots</code>

Appendix C - Glossary

Term	Meaning
Array	An ordered list of values, written <code>[1, 2, 3]</code> .

Term	Meaning
Callback	A function passed to another function to be called later.
Delta time	Seconds since the last frame, used to make movement frame-rate independent.
Event	Something the browser reports, such as a key press.
Hitbox	The rectangle used for collisions.
Hoisting	Function declarations being usable above where they are written.
Object	A bundle of named values, written <code>{ x: 1, y: 2 }</code> .
Procedural generation	Building content with randomness and rules.
Scope	The region of code where a name exists.
State machine	Logic with one current state and rules for changing it.
Template literal	A string in backticks that can include <code>\${value}</code> .
Ternary	The <code>a ? b : c</code> expression.

Appendix D - Answer key

Lesson 1

1. Names declared inside are private to the block, so they cannot clash with other scripts. Only the `window` assignments are visible to the page.
2. `getElementById` returns null, so `canvas.getContext` throws an error.

Lesson 2

1. `gameRunning` is reassigned during play; `TILE_SIZE` never changes.
2. $5.2 \times 0.016 = \text{about } 0.083 \text{ tiles, roughly } 2.7 \text{ pixels.}$

Lesson 3

1. The outer array holds rows, so the first index chooses the row (y) and the second the column (x).
2. A Set ignores duplicates and has fast `has` and `delete`, which suits "is this key down right now".

Lesson 4

1. `undefined`.
2. Function declarations are hoisted to the top of their scope; `const` arrow functions are not.

Lesson 5

1. It skips the rest of that pass, so wall tiles that do not border a room are not drawn.
2. `a && b`, because `&&` binds tighter than `||`.

Lesson 6

1. `keydown` fires once and then at the OS repeat rate; the game needs to know every frame whether the key is currently down.
2. It names the physical key, so it works on keyboard layouts where the letter is elsewhere.

Lesson 7

1. Speed is tiles per second and dt is seconds per frame, so the product is tiles per frame and the on-screen speed does not depend on frame rate.
2. 0.05 and about 0.021 tiles, both equal 3 tiles per second overall.

Lesson 8

1. The camera transform is undone, so the HUD stays fixed on screen instead of moving with the world.
2. Later drawing covers earlier drawing, so the background must go at the bottom.

Lesson 9

1. `find` returns the first match (or undefined); `filter` returns an array of every match.
2. `??` falls back only for null or undefined; `||` falls back for any falsy value, including 0 and an empty string.

Lesson 10

1. If one axis is blocked the other can still move, so the player slides instead of sticking.
2. Only the feet collide, so the head can overlap the wall behind (a top-down depth effect).

Lesson 11

1. It visits tiles in order of distance and every step costs one, so the first arrival at the goal is shortest.
2. It records each tile's parent; walking back from the goal rebuilds the path, which is then reversed.

Lesson 12

1. Random layouts can fail (for example too few rooms for the stairs, so `createGameState` returns null); trying again is cheap.
2. 35 out of 100, so 35 percent.

Lesson 13

1. Otherwise light would shine through walls and a player in the next room would count as lit.
2. It erases existing pixels where the new shape is drawn, so the gradient cuts holes in the dark layer.

Lesson 14

1. Alert enemies are already hunting you and use the longer `ALERT_SIGHT_RANGE`.
2. It is waist high, so a standing player is visible over it.

Lesson 15

1. It reuses your sound system, so every landing can attract enemies without new AI code.
2. The object lands at the mouse position unless that is farther than the throw range.

Lesson 16

1. Other enemies treat closed doors as walls and should not suddenly gain dungeon-wide movement.
2. The route choice uses `homePatrol` whenever the torch is missing or out.

Lesson 17

1. A tree has exactly one path between any two rooms; each extra connector between rooms already joined adds a loop.
2. To make sure the cover never blocks the only route, which would split the level.